REKOLT Planters' Cooperative Produce Tracker - Design v1

1. **Scope Statement with My Assumptions**
   Our System is a console-based Java application, designed to help users manage produce deliveries during a season. It records delivery details, calculates the payments for members based on our given rules, works out the required statistics for the season, and generates a word report for each member and all this data is stored in memory while the application is running.
   What we are not covering: Database/file storage, loading previous seasons, a graphical interface, multiple simultaneous users, and editing or deleting recorded deliveries.

   My Assumptions:
   1. A member's name does not change during a season.
   2. Delivery IDs are generated automatically.
   3. A season has a maximum of 20 weeks.
   4. The four produce codes are fixed.
   5. Monetary calculations are rounded only when displayed.
   6. REJECT deliveries remain in the season records and volume statistics but have zero payment.

2. **Numbered Functional Requirement**
   The following requirements are numbered so that each one can be checked against the final system.

   a. FR1- Record a Delivery

   The system will allow the user to record a delivery by entering:
   member ID, member name, produce code, mass in kg, quality score, week number
   A unique delivery ID shall be generated automatically.

   b. FR2- Validates delivery input

   The system will validate every delivery field before recording it. If an input is invalid, the system shall display an error message and allow the user to enter the correct value again rather than crashing.

   c. FR3- Grade deliveries

   The system will assign a grade to every recorded delivery according to its quality score boundary:
   85-100 gives an A, 70-84 gives a B, 50-69 gives a C, and 0-49 gives a REJECT.

   d. FR4- Calculate net payment

The system will calculate the net payment using the five specified calculation steps in the required order. A REJECT delivery will be assigned a net payment of MUR0.00 while it's still keeping that delivery in the season's records and volume statistics.

e. FR5- Display season information

The system will display the total payment for each member, a weekly volume grid, and the top 5 deliveries ranked by value.

f. FR6- Continue until exit

The system will continue accepting user choices and performing operations until the user selects the exit option.

3. Numbered Non-Functional Requirement

a. NFR1- Calculation Accuracy

All monetary calculations shall retain the full decimal precision during the calculation and should match our specified worked example when its being displayed to the user, exactly.

b. NFR2- Input Safety

Invalid user input shall not cause an unhandled program crash. The system will display an appropriate error message and allow the user to try again.

c. NFR3- Usability

The console shall clearly label input fields, menu options, and also calculated results so that a user will be able to understand what action is required.

4. Noun-Verb Analysis

The noun-verb analysis was used to identify the possible classes and methods from the requirements.

Nouns that were kept as classes:

- Member(Stores our member information)
- Produce(Stores our produce information)
- Delivery(Represents an induvidual delivery)
- Grade(Represents the four possible delivery grades and their muiltipliers)
- ProduceService(Provides access to the fixed produce information and the list of the prices)
- SeasonService(Manages season-level delivery records, statistics and searches)
- InputValidator(Handles the validation of user input)
- Main(Controls the console app and user interaction)

Verbs that were kept as methods:

- record: recordDelivery()
- grade(of a score)

- calculate: baseValue(), gradedValue(), categorisedValue(), commission(), transportLevy(), netPayable()
- sort(by value)
- search: findDeliveryById()
- remove: deliveriesWithRejectedRemoved()
- validate: methods in InputValidator
- display: handled by Main

Classes we rejected, and why:

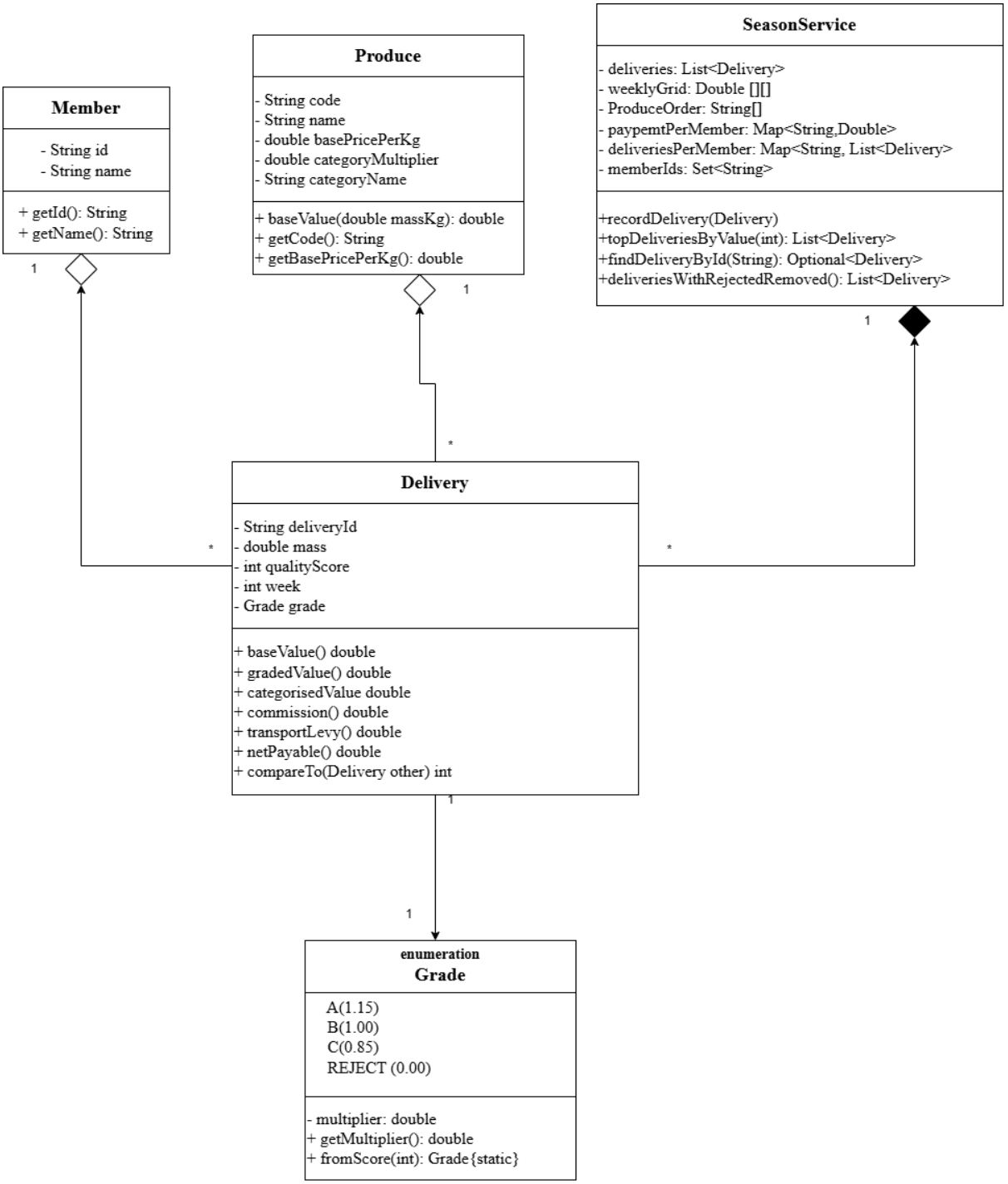
a. I considered using Season as a separate domain class, but the required season behaviour is limited to managing the deliveries, payment totals, weekly grid and member information. This can be handled by SeasonService, so creating both would be an unnecessary split which can more or less do the same responsibility.

b. I considered making ProduceService as an object that you create with new, but I rejected this in favour of static methods because there's only ever one fixed price list for this season, so me giving it instance state would let there be 2 different price list exist by accident, which is not needed here.

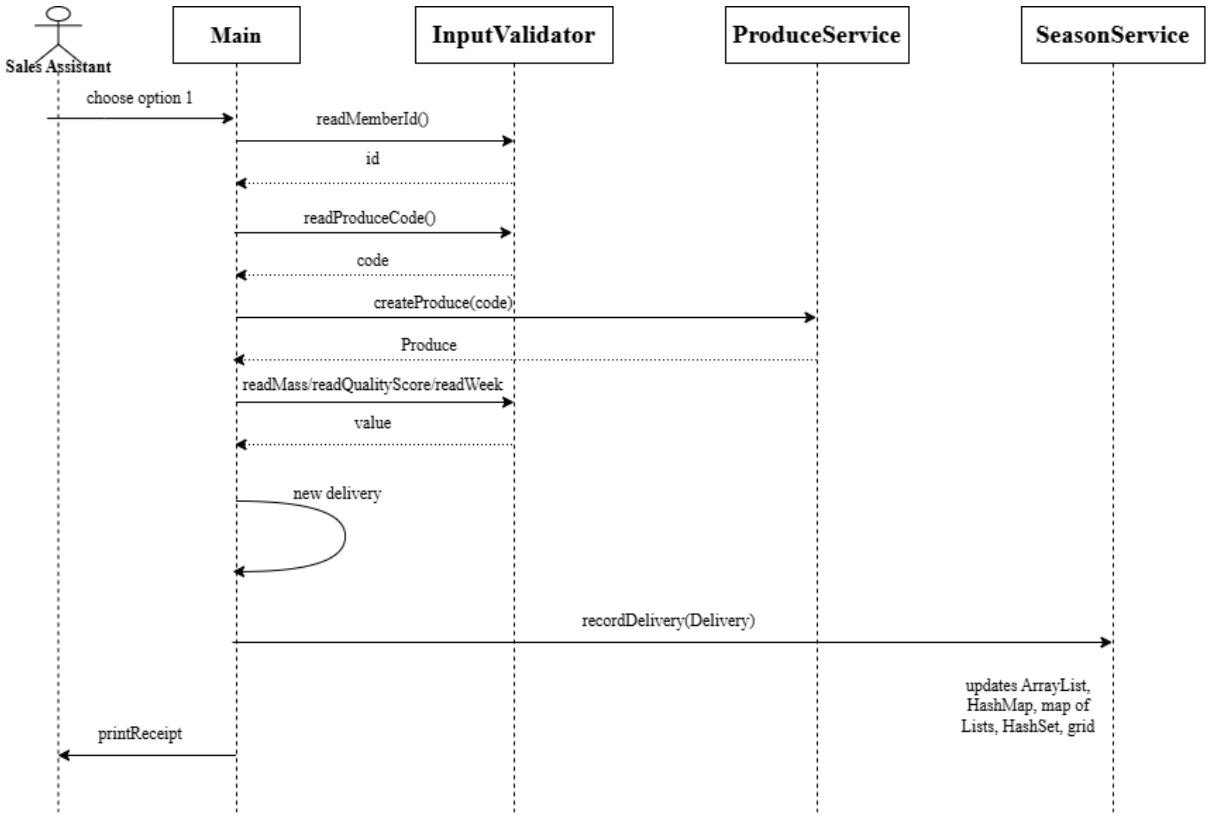
c. I rejected using Payment as a separate class from Delivery, because the payment values are already calculated directly from a delivery's mass, produce, grade, and other information, hence, creating a separate Payment class would would just add another object without actually providing behaviour that is required by the system.

5. UML class diagram

A UML class diagram with typed attributes, full method signatures and visibility markers, and relationships with multiplicities that distinguish inheritance, association, aggregation and composition.



6. Sequence Diagram



7. Pseudocode

```
FUNCTION netPayable(delivery):
    grade = gradeFromScore(delivery.qualityScore)

    IF grade == REJECT:
        RETURN 0.00
    END IF

    baseValue = delivery.mass * delivery.produce.basePricePerKg
    gradedValue = baseValue * grade.multiplier
    categorisedValue = gradedValue * delivery.produce.categoryMultiplier
    commission = categorisedValue * 0.05
    transportLevy = delivery.mass * 2.00
    payableAmount = categorisedValue - commission - transportLevy

    RETURN payableAmount
END FUNCTION

FUNCTION gradeFromScore(score):
    grade = REJECT
```

```
    IF score >= 85:
        grade = A
    ELSE IF score >= 70:
        grade = B
    ELSE IF score >= 50:
        grade = C
    END IF

    RETURN grade
END FUNCTION
```

8. Data Dictionary

a. Member.id

- Type: String
- Unit: No unit
- Range: M-####
- Validation Type: It must match M-\d{4}

b. Member.name

- Type: String
- Unit: No unit
- Range: text
- Validation Type: It must not be blank

c. Produce.code

- Type: String
- Unit: No unit
- Range: MZE, BNS, POT, TEA
- Validation Type: It must be one of these 4 codes

d. Delivery.mass

- Type: double
- Unit: kg
- Range: (0, 5000]
- Validation Type: It must be greater than 0 and less than or equal to 5000

e. Delivery.qualityScore

- Type: int
- Unit: score
- Range: [0, 100]
- Validation Type: It must be a whole number in this range

f. Delivery.week

- Type: int
- Unit: week number
- Range: [1, 20]
- Validation Type: It must be a whole number in this range

g. Delivery.grade
- Type: Grade
- Unit:No unit
- Range: A, B, C, REJECT
- Validation Type: This is derived from qualityScore

h. Delivery.netPayable()
- Type: double
- Unit: MUR
- Range: >= 0
- Validation Type: This is rounded on display

9. Rationale for 3 decisions

i. Validation in Delivery and Member: I kept validation inside Delivery and Member instead of relying only on the InputValidator. Though the InputValidator already checks the data before it reaches Main, and the constructor validates it again. I figured this may be repetitive but it makes sure that Delivery and Member cannot be created with invalid data from somewhere else in the system. The only alternative was to rely only on InputValidator, but then that would mean the classes are only safe when they are created through the console input process.

ii. Using Grade as an enum: Instead of storing the grade as a string or char and using separate if or switch statements to find its multiplier, I made grade an enum with its multiplier stored in each grade. So an example is, A(1.15) stores the value of grade A in one place. This avoids us repeating the same logic in different parts of the syste and makes the grade rules easier for us to maintain.

iii. Splitting the payment calculation: Instead of doing everything in one method, I decided to separate the payment calculation in different methods, such as baseValue(), gradedValue(), categorisedValue(), commission(), transportLevy(), and netPayable(). It makes the steps easier to understand and test.